SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1741**

Register Number	**192412093**
Name	**Naresh Kanna. C. M**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS:2 **Total Marks:20**

Descriptive Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behaviour and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.

   Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Understanding of Concepts	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	5
Explanation	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	5
Application	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	4
Organization	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	3
Accuracy	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	2
Clarity	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	1

1. Types of Intelligent Agents in a Smart Home Automation System

Introduction

An intelligent agent is a system that perceives its environment through sensors and takes actions through actuators to achieve specific objectives. In a smart home automation system, sensors collect information such as room temperature, motion, lighting conditions, and time of day. The system processes this information to control devices such as lights, air conditioners, alarms, and smart locks.

The four major types of intelligent agents are simple reflex agents, model-based reflex agents, goal-based agents, and utility-based agents. Each type has different capabilities for decision-making and problem-solving.

1.1 Simple Reflex Agent

A simple reflex agent selects actions based only on the current percept, using predefined condition-action rules. It does not consider past experiences or maintain an internal representation of the environment.

Working in a smart home:

- If motion is detected in a dark room, switch on the lights.
- If the temperature exceeds a specified limit, turn on the air conditioner.
- If a door sensor detects unauthorized entry, activate the alarm.

Advantages:

- Simple to design and implement.
- Responds quickly to real-time inputs.
- Requires relatively little computational power.

Disadvantages:

- Cannot remember previous states or learn user preferences.
- May make inappropriate decisions when the current input does not provide enough information.
- Cannot effectively handle complex situations requiring long-term planning.

Example: If a motion sensor detects movement at night, the agent immediately switches on the hallway light.

1.2 Model-Based Reflex Agent

A model-based reflex agent maintains an internal state that represents relevant aspects of the environment. It uses current sensor readings, previous observations, and a model of how the environment changes to make decisions.

Working in a smart home:

- Remembers whether a room is occupied.
- Tracks the current temperature and the recent operation of the air conditioner.
- Uses door and motion sensor history to distinguish normal household activity from potentially suspicious events.
- Maintains the current state of lights and appliances, even when some sensors provide incomplete information.

Advantages:

- Handles partially observable environments better than simple reflex agents.
- Uses historical information to improve decisions.
- Can avoid unnecessary switching of devices.

Disadvantages:

- Requires additional memory and a reliable internal model.
- Incorrect or outdated information can lead to poor decisions.
- A basic model-based agent does not necessarily learn preferences automatically.

Example: If a room becomes temporarily motionless while its occupant is reading, the agent may keep the lights on instead of switching them off immediately.

1.3 Goal-Based Agent

A goal-based agent selects actions by considering a desired future state. It evaluates possible actions and chooses those that help achieve its goals, often using search and planning techniques.

Working in a smart home:

- Maintains a goal of keeping the living room at a comfortable temperature.
- Plans to reduce energy consumption when nobody is home.
- Ensures that doors are locked and the security system is activated when the house is unoccupied.
- Coordinates heating, cooling, lighting, and security actions to achieve a desired household state.

Advantages:

- Supports planning and decision-making over multiple steps.
- Adapts actions to changing circumstances.
- Can consider several actions before selecting one.

Disadvantages:

- Requires clearly defined goals and a way to evaluate possible actions.
- Planning may consume considerable computational resources.
- Achieving a goal does not necessarily mean choosing the cheapest, safest, or most comfortable solution when several solutions are possible.

Example: When the occupants leave home, the agent checks occupancy, switches off unnecessary lights, adjusts the thermostat to an energy-saving setting, and activates the security system.

1.4 Utility-Based Agent

A utility-based agent evaluates possible outcomes using a utility function that assigns a numerical value to how desirable each outcome is. It selects actions that maximize expected utility while considering uncertainty and competing priorities.

Working in a smart home:

- Balances comfort against electricity consumption.
- Chooses an appropriate temperature without wasting energy.
- Adjusts lighting according to occupancy, daylight, and user preferences.
- Prioritizes security while avoiding unnecessary alarms.

For example, the agent may compare different air-conditioning settings based on comfort, electricity cost, and environmental conditions before selecting the most suitable option.

Advantages:

- Balances multiple objectives simultaneously.
- Supports trade-offs between comfort, cost, convenience, and safety.
- Can make decisions under uncertainty by considering expected outcomes.
- Can incorporate learned user preferences into its utility function.

Disadvantages:

- Designing a suitable utility function can be difficult.
- Assigning appropriate weights to comfort, cost, and security requires careful consideration.
- Incorrect sensor data or poorly chosen weights may produce undesirable decisions.

Example: During a hot afternoon, the agent may maintain a comfortable temperature while reducing cooling when the room is unoccupied, rather than running the air conditioner continuously.

1.5 Comparison of Intelligent Agents

Agent type	Decision-making method	Main advantage	Main limitation
Simple reflex	Current input and fixed rules	Fast response	No memory or planning
Model-based	Current input and internal state	Handles incomplete information	Depends on model accuracy
Goal-based	Actions that achieve desired goals	Supports planning	Goals may not express trade-offs
Utility-based	Maximizes the desirability of outcomes	Balances competing objectives	Utility design can be complex

1.6 Most Effective Approach for the Smart Home

A hybrid approach combining model-based, goal-based, and utility-based agents, supported by simple reflex rules for urgent events, would be the most effective solution.

The model-based component maintains information about room occupancy, temperature, device status, and recent activity. The goal-based component plans actions to achieve objectives such as maintaining comfort, reducing energy consumption, and securing the house. The utility-based component compares alternative actions and selects the best trade-off among these objectives.

Simple reflex rules can provide immediate responses to critical events, such as activating an alarm when a verified security condition occurs. These rules should operate alongside appropriate safety checks and emergency overrides.

The system can also learn user preferences from repeated behaviour, such as preferred room temperatures, lighting levels, and usual schedules. The learned preferences can update its model and utility calculations.

For example, if a family usually prefers a cool living room in the evening, the system can learn this pattern and prepare the room before the expected arrival time. If the home is empty, it can reduce cooling to save electricity while keeping security protections active.

Conclusion: A hybrid intelligent agent is preferable because no single agent type handles every requirement equally well. Combining rapid reactions, environmental memory, planning, utility optimization, and preference learning provides a balanced solution for comfort, energy efficiency, and security.

2. Uninformed Search Strategies for a Robot in an Unknown Maze

Introduction

A search algorithm is a problem-solving technique used by an intelligent agent to find a sequence of actions that leads from an initial state to a goal state. In an unknown maze, a robot must discover the available paths, avoid obstacles, and reach the exit without having prior knowledge of the complete layout.

Uninformed search strategies, also known as blind search strategies, explore possible states without using additional heuristic information about how close a state is to the goal. Two important strategies are Uniform Cost Search (UCS) and Iterative Deepening Search (IDS).

2.1 Uniform Cost Search (UCS)

Uniform Cost Search expands the node with the lowest cumulative path cost. It uses a priority queue to select the next node for exploration, ensuring that the least expensive path discovered so far is considered first.

Working of UCS in a maze:

1. The robot starts at its initial position and assigns it a path cost of zero.
2. It explores accessible neighbouring cells and calculates the cost of reaching each cell.
3. It stores the discovered cells in a priority queue, ordered by cumulative path cost.
4. It repeatedly expands the lowest-cost cell and updates path costs when a cheaper route is discovered.
5. The process continues until the exit is reached and selected for expansion.

For example, if one route to the exit contains many low-cost movements while another contains fewer but expensive movements, UCS compares their total costs and chooses the cheaper route.

Advantages:

- Complete when each action has a positive cost bounded away from zero and the branching factor is finite.
- Finds an optimal path when path costs are non-negative.
- Handles mazes in which different movements have different costs.

Disadvantages:

- Can consume substantial memory because it stores many frontier nodes.
- May explore many unnecessary cells before reaching the exit.
- Can be slow when the search space is large or when many paths have similar costs.

Complexity:

Let \(b\) be the branching factor, \(C^*\) the optimal solution cost, and \(\epsilon\) the minimum positive action cost.

- Time complexity: \(O(b^{1+\lfloor C^*/\epsilon\rfloor})\), commonly expressed as exponential in the optimal path cost.
- Space complexity: Exponential in the optimal path cost, because the priority queue and explored-node storage can become large.

These are general worst-case bounds; actual performance depends on the maze structure and path costs.

2.2 Iterative Deepening Search (IDS)

Iterative Deepening Search combines the depth-first exploration style of Depth-First Search (DFS) with the increasing depth limits of Breadth-First Search (BFS).

Instead of exploring the entire maze at once, IDS performs repeated depth-limited searches. It begins with a depth limit of zero and increases the limit by one after each unsuccessful search.

Working of IDS in a maze:

1. The robot begins at the starting cell with a depth limit of zero.
2. It searches for the exit within the current depth limit.
3. If the exit is not found, it increases the limit to one and repeats the search.
4. It continues with limits of two, three, four, and so on.
5. Once the exit is found, the robot returns the path discovered within the successful depth limit.

For example, if the exit is four movements away, IDS searches repeatedly with depth limits of zero, one, two, three, and four until it reaches the exit.

Advantages:

- Complete when the branching factor is finite and a reachable goal exists.
- Requires much less memory than UCS.
- Finds a shallowest solution when every movement has equal cost.
- Useful when the depth of the solution is unknown.

Disadvantages:

- Repeats the exploration of shallow nodes during each iteration.

- Can be inefficient if the solution is very deep.
- Does not guarantee the lowest-cost path when movement costs differ.
- A basic IDS implementation may revisit cells indefinitely in cyclic mazes unless it limits repeated states within each iteration or otherwise prevents unproductive cycles.

Complexity:

Let \(b\) be the branching factor and \(d\) the depth of the shallowest solution.

- Time complexity: \(O(b^d)\).
- Space complexity: \(O(bd)\) for a depth-limited DFS implementation that stores the current path and a bounded number of frontier nodes.

Although IDS repeats some work, most nodes occur at the deepest level, so the repeated searches generally retain the same asymptotic time complexity as BFS.

2.3 Comparison of UCS and IDS

Criterion	Uniform Cost Search	Iterative Deepening Search
Search method	Expands the lowest-cost frontier node	Repeats depth-limited DFS
Main priority	Cumulative path cost	Increasing search depth
Completeness	Yes, under suitable cost and branching assumptions	Yes, for a finite branching factor and reachable goal
Optimality	Yes, for non-negative costs under standard conditions	Yes, for equal step costs
Time complexity	Exponential in optimal path cost in the general case	O(b^d)
Space complexity	Exponential in the general case	O(bd)
Best use	Mazes with different movement costs	Mazes with equal movement costs and limited memory

2.4 Relationship Between Intelligent Agents and Maze Search

The robot's success depends not only on the search algorithm but also on the type of intelligent agent controlling its decisions.

Simple reflex agent: The robot responds to its current sensor readings using fixed rules, such as turning left when the left passage is open. This approach is fast but can fail in dead ends or cycles because it does not remember previously visited locations.

Model-based agent: The robot maintains an internal representation of discovered passages, walls, and visited cells. It can use this information to avoid repeating failed movements and make better decisions when parts of the maze remain unexplored.

Goal-based agent: The robot treats reaching the exit as its goal and searches for a sequence of movements that leads to it. UCS and IDS can both be used as its planning algorithms.

Utility-based agent: The robot evaluates paths according to their desirability, considering factors such as distance, battery consumption, risk, and travel time. A cost-based search strategy can help it choose a route that minimizes the relevant costs, provided those costs are represented appropriately.

An important distinction is that the agent type describes how decisions are made, while the search strategy describes how possible paths are explored. Therefore, an intelligent robot can use a model-based agent with UCS or IDS, depending on the environment and the objective.

2.5 Most Effective Combination for the Maze Robot

A model-based, goal-based agent using Uniform Cost Search is a strong choice when the maze is initially unknown and different movements may have different costs.

The model-based component records the information obtained from sensors, including discovered walls, accessible passages, and visited locations. The goal-based component defines reaching the exit as the objective. UCS then evaluates alternative routes using their cumulative costs and selects the least-cost path among those discovered.

The robot should follow a repeated process of sensing, updating its internal map, planning, and moving. Since the maze is initially unknown, it cannot plan a complete route immediately. It must discover the environment incrementally and replan when new information changes the available routes.

For example, suppose the robot discovers two routes:

- Route A: six movements with a cost of one unit per movement, giving a total cost of six.
- Route B: four movements with a cost of two units per movement, giving a total cost of eight.

Although Route B is shorter in terms of movement count, UCS selects Route A because its total cost is lower.

If every movement has the same cost and the robot has very limited memory, IDS is a suitable alternative because it finds a shallowest solution while using substantially less memory than UCS.

For a robot that must also account for uncertain observations, battery limitations, and changing obstacles, a hybrid model-based and goal-based design can be extended with utility-based decision-making. In such a system, the utility function can represent energy consumption, route length, and safety risks, while the search algorithm evaluates candidate paths.

Conclusion: The most appropriate approach depends on the maze's cost structure and the robot's memory constraints. Model-based and goal-based agents provide reliable environmental reasoning and planning. UCS is preferable when minimizing total path cost is important, whereas IDS is preferable when movements have equal costs and memory is limited. Combining an appropriate agent design with a suitable search strategy improves efficiency, reliability, and the robot's ability to reach the exit.